

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 10

Course: Operating Systems

Topic: Add Your Own System Call: trace

Done by:
st. gr. FAF-243, Ilie Covali

Checked by:
Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 10 · Add Your Own System Call: trace

Metadata & Environment

- **Course:** Operating Systems, FCIM / FAF, UTM, Academic Year 2026-2027
- **Lab Work:** Lab 10 (Track B: Add Your Own System Call: trace, xv6)
- **Author:** Ilie Covali (st. gr. FAF-243)
- **Verified by:** Reițman Patricia
- **Host OS:** Linux (CachyOS x86_64, Kernel 7.3.1-1-cachyos)
- **Target Architecture:** RISC-V 64-bit (`riscv64-elf-gcc` 14.2.0)
- **Emulator:** QEMU System Emulator (`qemu-system-riscv64` 10.1.0)
- **Source Tree:** `/home/ilie/xv6-riscv`
- **Workspace:** `/home/ilie/Documents/SO/os-lab10` (`~/os-lab10`)

Purpose of the Work

1. Understand the complete journey of a system call from user space across the hardware privilege boundary into the operating system kernel on the RISC-V 64-bit architecture.
2. Extend the xv6 RISC-V operating system kernel with a custom system call, `trace` (syscall number 23), enabling user-controlled selective interception and logging of system call executions.
3. Implement user-space wrappers, automatic assembly stub generation (`user/usys.pl`), and the user-level utility binary `user/trace.c`.
4. Augment the process control block (`struct proc`) to maintain a persistent per-process tracing bitmask (`trace_mask`).
5. Develop the kernel-space system call handler (`sys_trace`), extracting register arguments via `argint()`, and implement dynamic call interception in `syscall()`.
6. Maintain process tracing state across process replication (`fork()`), verifying that child processes inherit the tracing mask.
7. Experiment with selective bitmask configurations, verifying bitwise filtering of system call numbers and analyzing the precise timing of kernel trace emissions relative to standard I/O buffer flushes.

Part 1. The Number and the User Side

1. The Anatomy of an xv6 System Call

In the RISC-V architecture, user-space code executes in unprivileged User Mode (U-mode). To request services from the operating system kernel executing in Supervisor Mode (S-mode), a user program must perform the following steps: 1. Place the unique system call number into register `a7`. 2. Place system call arguments into registers `a0`, `a1`, `a2`, etc. 3. Execute the `ecall` (environment call) instruction. 4. The CPU hardware switches privilege mode to S-mode, saves user registers in the process trapframe, and transfers control to the kernel trap vector (`kernel/trampoline.S`). 5. After the kernel executes the requested handler, it places the return code into register `a0`, restores the user registers, and executes `sret` to return to U-mode.

In xv6, user-space programs do not write bare assembly for every call. Instead, the Perl script `user/usys.pl` automatically emits assembly stubs into `user/usys.S`:

```
.global trace
trace:
 li a7, SYS_trace
 ecall
 ret
```

Figure 1.1 – Assembly stub generated by user/usys.pl for the trace system call

2. Implementation: The User-Side Edits

To introduce the `trace` call, we modify five files across the user-space interface:

1. Assign System Call Number (`kernel/syscall.h`) We define `SYS_trace` as 23 (preserving `SYS_sync` at 22):

```
#define SYS_sync   22
#define SYS_trace  23
```

2. Declare User Function Prototype (`user/user.h`) We add the function prototype accessible to user applications:

```
int sync(void);
int trace(int);
```

3. Update Assembly Generator (`user/usys.pl`) We append the entry generation rule:

```
entry("sync");
entry("trace");
```

4. Implement User Utility (`user/trace.c`) The `trace` program parses a numeric bitmask, installs it in the calling process via `trace(mask)`, and replaces its process image with the target command using `exec()`:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

Figure 1.2 – Implementation of user/trace.c

5. Register Program in Disk Image (`Makefile`) In the `UPROGS` target list, we add `$U/_trace\` with a leading tab:

```
    $U/_zombie\
    $U/_trace\
```

3. Check 1: Verifying Trap Delivery to Kernel

Before implementing the kernel handler, we compile and boot xv6 in QEMU to ensure that the user-space stub successfully issues the `ecall` trap with `a7 = 23`:

```
$ trace 32 grep hello README
3 trace: unknown sys call 23
trace: failed
```

Figure 1.3 – Check 1: The kernel receives syscall 23 and reports unknown handler

This message confirms that the user-space stub loaded register `a7` with 23, executed `ecall`, trapped into `kernel/syscall.c:syscall()`, and reached the default branch of the dispatch table.

Part 2. The Kernel Side

1. Extending the Process Control Block (`kernel/proc.h`)

Each process must independently track its own active tracing mask. In `kernel/proc.h`, inside `struct proc`, we add the integer field `trace_mask`:

```
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
  int trace_mask;              // which syscalls to print
};
```

Figure 2.1 – Adding trace_mask to struct proc in kernel/proc.h

In `kernel/proc.c:freeproc()`, we also explicitly reset `p->trace_mask = 0;` upon process cleanup.

2. Implementing the Handler (`kernel/sysproc.c`)

The handler function `sys_trace()` extracts its single integer argument from register `a0` using the kernel helper `argint()` and stores it in the current process's PCB (`myproc()->trace_mask`):

```
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

Figure 2.2 – Implementation of sys_trace in kernel/sysproc.c

3. Registering the Dispatch Handler (`kernel/syscall.c`)

We declare `sys_trace` as an external function and insert it into the function pointer dispatch array `syscalls[]` at index `SYS_trace`:

```
extern uint64 sys_close(void);
extern uint64 sys_sync(void);
extern uint64 sys_trace(void);

static uint64 (*syscalls[])(void) = {
  ...
  [SYS_close]   = sys_close,
  [SYS_sync]    = sys_sync,
  [SYS_trace]   = sys_trace,
};
```

4. Mapping Names and Intercepting System Calls (`kernel/syscall.c`)

Because numbers alone are not descriptive, we create a string table `syscallnames[]` indexed by system call numbers directly above the `syscall()` dispatch function:

```
static char *syscallnames[] = {
   [SYS_fork] "fork",   [SYS_exit] "exit",     [SYS_wait] "wait",
   [SYS_pipe] "pipe",   [SYS_read] "read",     [SYS_kill] "kill",
   [SYS_exec] "exec",   [SYS_fstat] "fstat",   [SYS_chdir] "chdir",
   [SYS_dup] "dup",     [SYS_getpid] "getpid", [SYS_sbrk] "sbrk",
   [SYS_pause] "pause", [SYS_uptime] "uptime", [SYS_open] "open",
   [SYS_write] "write", [SYS_mknod] "mknod",   [SYS_unlink] "unlink",
   [SYS_link] "link",   [SYS_mkdir] "mkdir",   [SYS_close] "close",
   [SYS_sync] "sync",   [SYS_trace] "trace",
};
```

Figure 2.3 – String lookup table for xv6 system calls

Inside `syscall()`, immediately after the dispatch function executes and stores its return value in `p->trapframe->a0`, we test whether bit `num` is active in `p->trace_mask`. If set, we emit the trace line using the kernel console printer `printk()`:

```
p->trapframe->a0 = syscalls[num]();
if ((p->trace_mask >> num) & 1)
  printk("%d: syscall %s -> %ld\n",
         p->pid, syscallnames[num], p->trapframe->a0);
```

Figure 2.4 – Interception and logging logic placed after syscall dispatch

5. Propagating the Mask Across Fork (`kernel/proc.c`)

A command invoked via `trace` often spawns child processes (or is itself exec'd by the shell). To ensure trace continuity across the process hierarchy, `fork()` must propagate the mask from parent to child. In `kernel/proc.c:fork()`, right after copying the process name, we copy `trace_mask`:

```
safestrcpy(np->name, p->name, sizeof(p->name));
np->trace_mask = p->trace_mask;
```

6. Check 2: Verifying Trace Execution

We compile and boot xv6 in QEMU to execute Check 2:

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

Figure 2.5 – Check 2 execution trace: read calls logged during grep execution

The observed byte counts match the exact file size of `README`:

$$1023 + 965 + 453 = 2441 \text{ bytes}$$

The final `read -> 0` indicates the end-of-file condition. Check 2 passes completely.

Part 3. Play with the Mask

1. Tracing All System Calls (`mask = 2147483647`)

The value 2147483647 is $2^{31} - 1$ (`0x7FFFFFFF`), which sets all 31 lower bits to 1. Executing `trace` with this mask logs every single system call invoked by `grep`:

```
$ trace 2147483647 grep hello README
3: syscall trace -> 0
3: syscall exec -> 3
3: syscall open -> 3
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
3: syscall close -> 0
```

Figure 3.1 – Complete system call sequence of grep hello README

Every lifecycle transition is captured: 1. `trace` configures the mask in the parent process. 2. `exec` replaces the process image with `/grep`. 3. `open` opens `README` and obtains file descriptor 3. 4. `read` ingests the file contents across three iterations and detects EOF on the fourth. 5. `close` releases file descriptor 3.

2. Tracing Fork Across the Process Tree (`mask = 2`)

Bit 1 corresponds to $2^1 = 2$, which is `SYS_fork`. We run `usertests forkforkfork`, which tests recursive fork creation until the process table (`proc[NPROC]`) is exhausted:

```
$ trace 2 usertests forkforkfork
usertests starting
test forkforkfork: 3: syscall fork -> 4
4: syscall fork -> 5
5: syscall fork -> 6
5: syscall fork -> 7
6: syscall fork -> 9
...
6: syscall fork -> 63
5: syscall fork -> 64
6: syscall fork -> -1
5: syscall fork -> -1
OK
```

Figure 3.2 – Recursive fork tracking across multiple PIDs

Notice that fork calls are emitted by multiple PIDs (PID 3, 4, 5, 6, etc.). When the process table fills up (64 processes), `fork()` returns `-1`, indicating failure, and the test exits successfully with `OK`.

3. Tracing Write and I/O Interleaving (`mask = 65536`)

Bit 16 corresponds to $2^{16} = 65536$, which is `SYS_write`. We execute `trace 65536 echo hi`:

```
$ trace 65536 echo hi
hi4: syscall write -> 2

4: syscall write -> 1
```

Figure 3.3 – Interleaved output from user write and kernel printk

Observe (Part 3 Answers)

1. **In `trace 32 grep hello README` why do you see only `read`, and not `open` or `close`?**
   The mask integer is interpreted as a binary bitfield where bit N controls tracing for system call number N. The decimal number 32 in binary is 100000_2 (2^5), which means only bit 5 is enabled. In `kernel/syscall.h`, `SYS_read` is defined as 5, whereas `SYS_open` is 15 and `SYS_close` is 21. Because bits 15 and 21 are 0, the kernel conditional `((p->trace_mask >> num) & 1)` evaluates to true solely when `num == 5` (`SYS_read`).

2. **In `forkforkfork`, three different PIDs print lines. Which line of code from Part 2 makes that possible?**
   The assignment statement in `kernel/proc.c` inside `fork()`:

   ```
   np->trace_mask = p->trace_mask;
   ```

   When a process forks, this line copies the parent's `trace_mask` directly into the child process's control block (`struct proc`). Because every descendant inherits this mask, subsequent `fork()` invocations made by children (e.g., PID 4, 5, 6) continue to satisfy the bitmask test and emit trace lines.

3. **In `trace 65536 echo hi` the trace line is printed between "hi" and the newline. Why? (How many times does echo call write?)**
   The `echo` utility invokes the `write` system call **twice**:

   - Call 1: Emits the argument string: `write(1, "hi", 2)`.
   - Call 2: Emits the trailing newline character: `write(1, "\n", 1)`.

   In `kernel/syscall.c`, the trace message is emitted by `printk()` immediately after each individual system call handler finishes:

   ```
   p->trapframe->a0 = syscalls[num]();
   if ((p->trace_mask >> num) & 1)
     printk("%d: syscall %s -> %ld\n", ...);
   ```

   Consequently, execution proceeds in strict chronological order:

   1. `echo` calls `write(1, "hi", 2)`, which places `"hi"` onto the UART transmitter.
   2. The kernel immediately emits the first trace message: `4: syscall write -> 2\n`.

3. `echo` calls `write(1, "\n", 1)`, transmitting the newline.
4. The kernel immediately emits the second trace message: `4: syscall write -> 1\n`.

4. **Which mask traces both `open` (15) and `close` (21), and nothing else?**
   To trace only system calls 15 and 21, bits 15 and 21 must be set to 1 while all other bits remain 0:

$$\text{Mask} = 2^{15} + 2^{21} = 32768 + 2097152 = \mathbf{2129920} \quad \text{(hex: 0x208000)}$$

   Verification in QEMU:

```
$ trace 2129920 grep hello README
3: syscall open -> 3
3: syscall close -> 0
```

   Only `open` and `close` are printed, confirming exact bitwise filtering.

Evaluation Criteria Compliance

Criterion	Target Points	Status	Verification & Evidence
Part 1: Check 1 reached (`unknown sys call 23`)	3 p	Met	User stub, syscall number 23, and `trace.c` created; verified with QEMU running `trace 32 grep hello README`.
Part 2: `trace 32 grep` prints the read lines	4 p	Met	Implemented `sys_trace()`, `trace_mask` in PCB, inheritance in `fork()`, and `printk` in `syscall()`; verified exact byte sequence ($1023 + 965 + 453 = 2441$).
Part 3: The three runs, Observe answered	3 p	Met	Executed all three test runs (mask 2147483647, mask 2 with `forkforkfork`, mask 65536 with `echo`); answered all 4 Observe questions in `notes.txt` and `notes.md`.
Source Tree: Modified xv6 committed to git	Required	Met	Committed to `/home/ilie/xv6-riscv` repository on branch `riscv` (commit `4190578`).
Total	**10 p**	**Met**	**Full score target satisfied.**

Conclusions

In this laboratory work, we extended the xv6 educational operating system kernel by implementing a custom system call, `trace` (SYS_trace 23), mirroring the functionality of the Linux `strace` tool investigated in Lab 9.

We traced the complete path of a system call across architectural boundaries: starting from user-space assembly stubs generated by `user/usys.pl`, transitioning into Supervisor Mode via the RISC-V `ecall` instruction, dispatching through `kernel/syscall.c`, and executing the handler `sys_trace()` in `kernel/sysproc.c`.

By adding `trace_mask` to `struct proc` and propagating it in `kernel/proc.c:fork()`, we demonstrated how kernel abstractions maintain process state and enable inherited diagnostic instrumentation across process hierarchies. Finally, through bitmask manipulation, we verified the bitwise mechanics of system call selection and gained a precise understanding of the interleaving between userspace I/O operations and kernel diagnostics.